

Lab 11: Datapath Design (Register File)

Objectives: Datapath Design (Register File)

After completing this lab, you will:

- Learn how to build a simple register file.

Flip-Flops and Latches

Flip-flops and latches are the simplest memory elements. In both flip-flops and latches, the output is equal to the value of the stored state inside the element. All the latches and flip-flops we will use from this point on are clocked, which means that they have a clock input and the change of state is triggered by that clock. The difference between a flip-flop and a latch is the point at which the clock causes the state to actually change. In a clocked latch, the state is changed whenever the appropriate inputs change and the clock is asserted, whereas in a flip-flop, the state is changed only on a clock edge. Since throughout this text we use an edge-triggered timing methodology where state is only updated on clock edges, we need only use flip-flops. Flip-flops are often built from latches, so we start by describing the operation of a simple clocked latch and then discuss the operation of a flip-flop constructed from that latch.

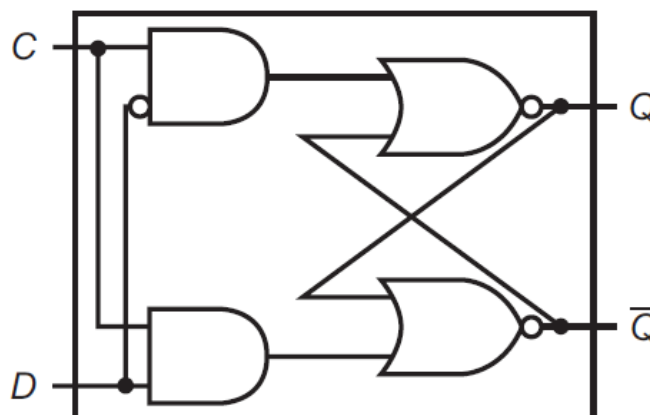


Figure 1: D latch implemented with NOR gates

For computer applications, the function of both flip-flops and latches is to store a signal. A D latch or D flip-flop stores the value of its data input signal in the internal memory. Although there are many other types of latch and flip-flop, the D type is the only basic building block that we

will need. A D latch has two inputs and two outputs. The inputs are the data value to be stored (called D) and a clock signal (called C) that indicates when the latch should read the value on the D input and store it. The outputs are simply the value of the internal state (Q) and its complement ($\bar{Q}$). When the clock input C is asserted, the latch is said to be open, and the value of the output (Q) becomes the value of the input D. When the clock input C is deasserted, the latch is said to be closed, and the value of the output (Q) is whatever value was stored the last time the latch was open. Figure 1 shows how a D latch can be implemented with two additional gates added to the cross-coupled NOR gates. Since when the latch is open the value of Q changes as D changes, this structure is sometimes called a transparent latch.

As mentioned earlier, we use flip-flops as the basic building block, rather than latches. Flip-flops are not transparent: their outputs change only on the clock edge. A flip-flop can be built so that it triggers on either the rising (positive) or falling (negative) clock edge; for our designs we can use either type. Figure 2 shows how a falling-edge D flip-flop is constructed from a pair of D latches. In a D flipflop, the output is stored when the clock edge occurs. The first latch, called the master, is open and follows the input D when the clock input, C, is asserted. When the clock input, C, falls, the first latch is closed, but the second latch, called the slave, is open and gets its input from the output of the master latch.

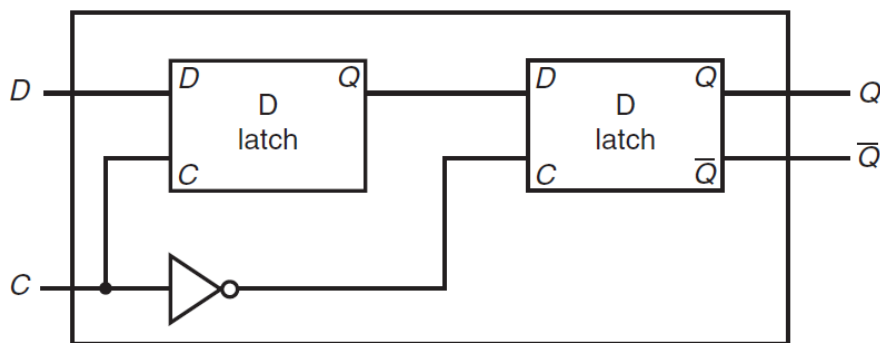


Figure 2: D flip-flop with a falling-edge trigger

We can use an array of D flip-flops to build a register that can hold a multibit datum, such as a byte or word.

Register Files

One structure that is central to our datapath is a register file. A register file consists of a set of registers that can be read and written by supplying a register number to be accessed. A register file can be implemented with a decoder for each read or write port and an array of registers built from D flip-flops. Because reading a register does not change any state, we only need to supply a register number as an input, and the only output will be the data contained in that register. For writing a register we will need three inputs: a register number, the data to write, and a clock that controls the writing into the register. In the lecture, we used a register file that has two read ports and one write port. This register file is drawn as shown in Figure 3. The read ports can be implemented with a pair of multiplexors, each of which is as wide as the number of bits in each register of the register file. Figure 4 shows the implementation of two register read ports for a 32-bit-wide register file.

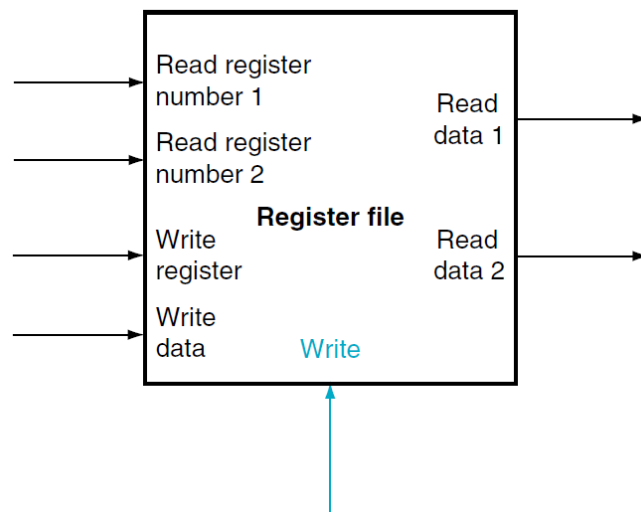


Figure 3: Simple Register File

Implementing the write port is slightly more complex, since we can only change the contents of the designated register. We can do this by using a decoder to generate a signal that can be used to determine which register to write. Figure 5 shows how to implement the write port for a register file. It is important to remember that the flip-flop changes state only on the clock edge. In the lecture, we hooked up write signals for the register file explicitly and assumed the clock shown in Figure 5 is attached implicitly.

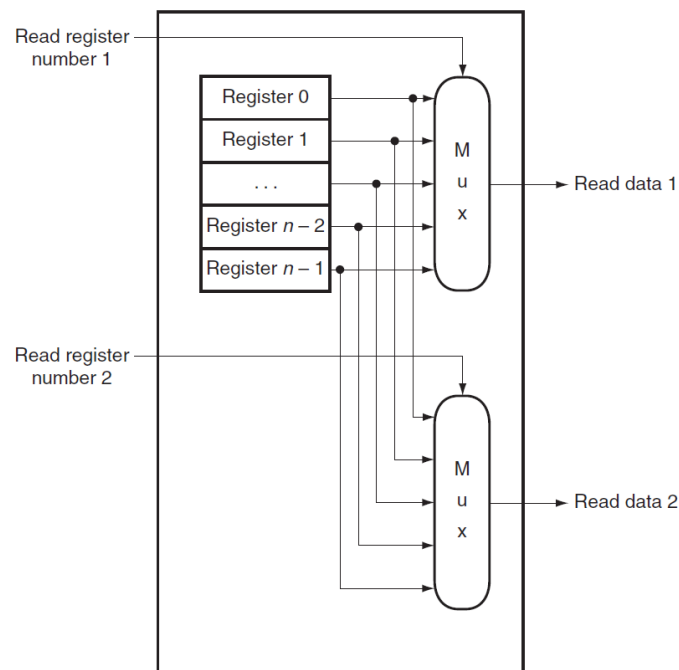


Figure 4: Two read-registers Register File

What happens if the same register is read and written during a clock cycle? Because the write of the register file occurs on the clock edge, the register will be valid during the time it is read. The value returned will be the value written in an earlier clock cycle. If we want a read to return the value currently being written, additional logic in the register file or outside of it is needed.

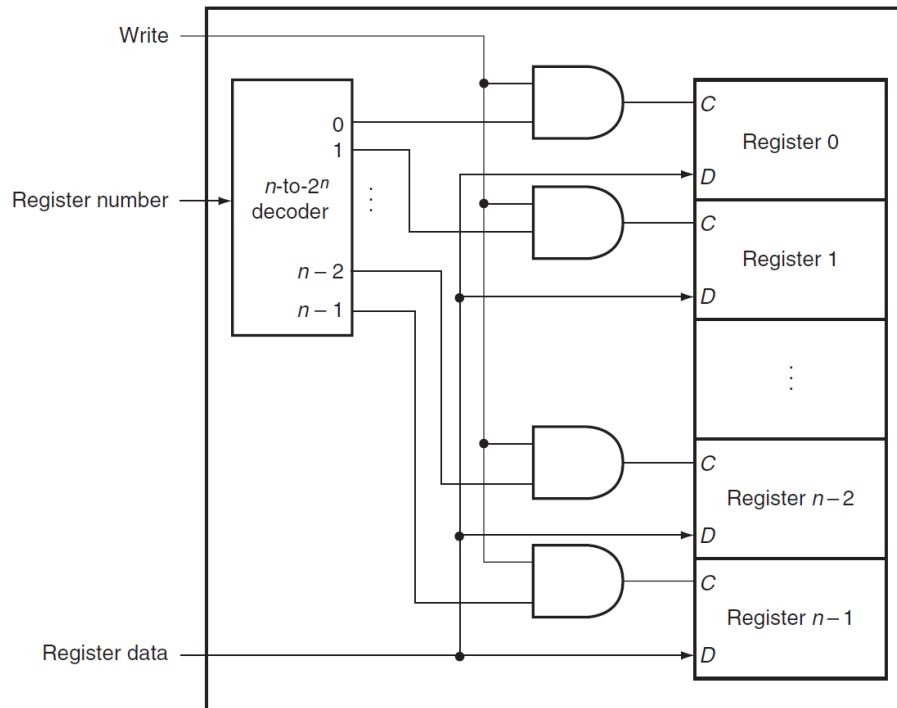


Figure 5: Data write Decoder

Assignment

1. Build a D Latch similar to Figure 1 in a sub-circuit named “DLatch”.
2. Use this latch to build a D Flip-Flop similar to Figure 2 in a sub-circuit named “DFlipFlop”.
3. Build a 32-bit Register using the “DFlipFlop” you built in a sub-circuit named “Register”.
4. Use 32 of those registers to build a Register File following the design in Figures 3, 4, and 5.

You have one week to complete this lab. Meaning, you have to return it before next week’s session.